

3. Funciones y operadores

Estas son las funciones y operadores que incluye R en su instalación básica.

3.1. Funciones principales

Función	Código en R
Raíz cuadrada de x	<code>sqrt(x)</code>
Exponencial de x	<code>exp(x)</code>
Logaritmo neperiano	<code>log(x)</code>
Nº de elementos de un vector	<code>length(x)</code>
Suma los elementos del vector	<code>sum(x)</code>
Seno de x	<code>sin(x)</code>
Coseno de x	<code>cos(x)</code>
Tangente de x	<code>tan(x)</code>
Media	<code>mean(x)</code>
Desviación típica	<code>sd(x)</code>
Varianza	<code>var(x)</code>
Mediana	<code>median(x)</code>
Quantiles	<code>quantile(x, p)</code>
Máximo y Mínimo	<code>range(x)</code>
Ordenación	<code>sort(x)</code>
Resumen estadístico	<code>summary(x)</code>

Ejemplos:

```
x = 16
sqrt(x)
```

```
## [1] 4
```

```
exp(x)
```

```
## [1] 8886111
```

```
mivector = c(3,6,2,7)  
length(mivector)
```

```
## [1] 4
```

```
mean(mivector)
```

```
## [1] 4.5
```

Operadores en R

Aritméticos

Operador	Descripción
+	Suma
-	Resta
*	Multiplicación
/	División
^	Potencia
%%	División entera

Ejemplos:

```
# Definición de variables
a <- 10
b <- 3

# Resultados de las operaciones
resultado_suma    <- a + b
resultado_resta    <- a - b
resultado_multi    <- a * b
resultado_div       <- a / b
resultado_pot       <- a ^ b
resultado_entera    <- a %% b

# Imprimir solo los resultados
print(resultado_suma)
```

```
## [1] 13
```

```
print(resultado_resta)
```

```
## [1] 7
```

```
print(resultado_multi)
```

```
## [1] 30
```

```
print(resultado_div)
```

```
## [1] 3.333333
```

```
print(resultado_pot)
```

```
## [1] 1000
```

```
print(resultado_entera)
```

```
## [1] 3
```

Comparativos

Operador	Descripción
==	Igualdad

Operador	Descripción
!=	Diferente de
<	Menor que
>	Mayor que
<=	Menor o igual
>=	Mayor o igual

Ejemplo:

```
# Definimos valores para comparar
x <- 10
y <- 5

# 1. Igualdad (==)
print(x == y)      # ¿Es 10 igual a 5? (FALSE)
```

```
## [1] FALSE
```

```
# 2. Diferente de (!=)
print(x != y)      # ¿Es 10 diferente de 5? (TRUE)
```

```
## [1] TRUE
```

```
# 3. Menor que (<)
print(x < y)        # ¿Es 10 menor que 5? (FALSE)
```

```
## [1] FALSE
```

```
# 4. Mayor que (>)
print(x > y)        # ¿Es 10 mayor que 5? (TRUE)
```

```
## [1] TRUE
```

```
# 5. Menor o igual (<=)
print(x <= 10)      # ¿Es 10 menor o igual a 10? (TRUE)
```

```
## [1] TRUE
```

```
# 6. Mayor o igual (>=)
print(y >= 15)      # ¿Es 5 mayor o igual a 15? (FALSE)
```

```
## [1] FALSE
```

Lógicos

Operador	Descripción
&	Y lógico
	O lógico
!	No lógico

Ejemplos:

```
x <- 10
y <- 5

# 1. Y lógico (&)
# ¿Es x mayor que 5 Y es y menor que 10?
print(x > 5 & y < 10) # TRUE porque ambas se cumplen
```

```
## [1] TRUE
```

```
# 2. O lógico (/)
# ¿Es x igual a 100 O es y igual a 5?
print(x == 100 | y == 5) # TRUE porque al menos una se cumple
```

```
## [1] TRUE
```

```
# 3. NO lógico (!)
# ¿Es x diferente de 10? (No, es igual, así que dará FALSE)
print(!(x == 10))
```

```
## [1] FALSE
```

Ejemplos con operadores

```
# Creamos la list/vector `datos`
datos <- c(4, 5, 10, 3)

min(datos)
```

```
## [1] 3
```

```
max(datos)
```

```
## [1] 10
```

```
which.min(datos)
```

```
## [1] 4
```

```
which.max(datos)
```

```
## [1] 3
```

```
range(datos)
```

```
## [1] 3 10
```

```
mean(datos)
```

```
## [1] 5.5
```

```
median(datos)
```

```
## [1] 4.5
```

```
sum(datos)
```

```
## [1] 22
```

```
2 * datos
```

```
## [1] 8 10 20 6
```

```
2 < 3 & 4 < 1
```

```
## [1] FALSE
```

```
2 < 3 | 4 < 1
```

```
## [1] TRUE
```

Asignación de variables

```
# Asignación de valores individuales a una variable  
a = 45  
b =5  
nombre = "Nicolás"  
  
print(a)
```

```
## [1] 45
```

```
#Asignación de operaciones a una variable  
resultado_suma      <- 5 + 3  
resultado_resta     <- 10 - 2  
resultado_multiplicacion <- 4 * 6  
resultado_division  <- 20 / 5  
resultado_potenciacion <- 2^3  
  
print(resultado_suma)
```

```
## [1] 8
```

Tipos de datos básicos

```
x1 <- TRUE                # logical  
x2 <- 3L                  # integer  
x3 <- 3.4                  # numeric  
x5 <- "Esto es una cadena de caracteres" # character  
  
ls.str()
```

```
## a : num 45
## b : num 5
## datos : num [1:4] 4 5 10 3
## mivector : num [1:4] 3 6 2 7
## nombre : chr "Nicolás"
## resultado_div : num 3.33
## resultado_division : num 4
## resultado_entera : num 3
## resultado_multi : num 30
## resultado_multiplicacion : num 24
## resultado_pot : num 1000
## resultado_potenciacion : num 8
## resultado_resta : num 8
## resultado_suma : num 8
## x : num 10
## x1 : logi TRUE
## x2 : int 3
## x3 : num 3.4
## x5 : chr "Esto es una cadena de caracteres"
## y : num 5
```

```
# Conversiones
as.integer(x1)
```

```
## [1] 1
```

```
as.integer(x3)
```

```
## [1] 3
```

```
# Comprobaciones
is.integer(x1)
```

```
## [1] FALSE
```

```
is.logical(x1)
```

```
## [1] TRUE
```

```
is.integer(x2)
```

```
## [1] TRUE
```


